CT Image Quality Assessment Using Phantom Experiment: Quantitative Analysis of SNR and CNR

1. INTRODUCTION AND THEORY

1.1 Backgrounds

This project focuses on the analysis of Signal-to-Noise Ratio (SNR) and Contrast-to-Noise Ratio (CNR) in computed tomography (CT) imaging. These two parameters are essential for evaluating image quality. SNR quantifies how strong the signal is compared to the background noise, thus reflecting the clarity of the image. CNR, on the other hand, takes into account the contrast between different tissues or structures and the surrounding noise level, offering a more comprehensive assessment of image detail and separation (**Fig. 1**).

In this study, a cone-shaped finger phantom was used to simulate a simple anatomical structure. The focus was placed on evaluating the SNR and CNR within selected regions of interest (ROIs) in the reconstructed CT images (**Fig. 2**). By computing the mean attenuation coefficients and associated noise levels in both object and background regions, we aim to quantitatively assess how well different structures can be distinguished.

1.2 Theory of SNR and CNR

Specifically, the formula for SNR is given by:

$$SNR = \frac{\mu_o}{\sigma_o} \tag{1}$$

where μ_o is the mean attenuation coefficient of the object within the ROI, and σ_o represents the noise, expressed as the standard deviation of pixel values in that region.

The formula for Contrast-to-Noise Ratio is:

$$CNR = \frac{\mu_o - \mu_B}{\sqrt{\sigma_o^2 + \sigma_B^2}} \tag{2}$$

where μ_B is the mean attenuation coefficient of the background, and σ_B is the standard deviation representing background noise. This metric gives a clearer indication of how distinguishable an object is from its surroundings.

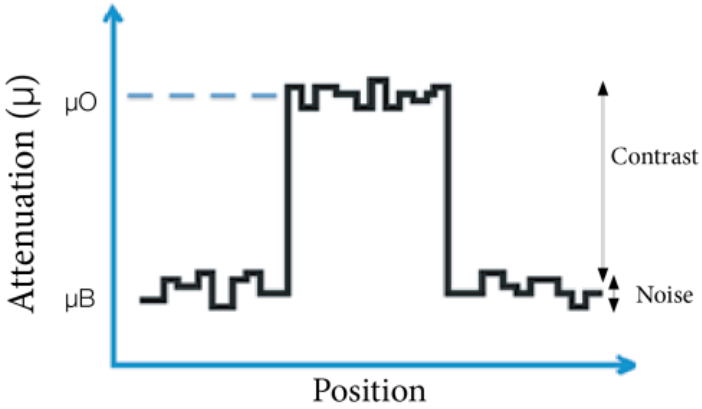

Fig 1. *Plot of CT attenuation coefficient (μ) across an object of interest (O) surrounded by a background (B). The contrast is the difference in the average attenuation coefficients. The background noise is the standard deviation in the value of* μ_B*. Higher noise levels may still allow for an accurate diagnosis provided the noise level is not too excessive in comparison with contrast* ($\Delta\mu >> \sigma_B$) *of a target region of interest*

2. DATA ACQUISITION

2.1 Scan experiment for data acquisition

To simulate the optical properties of biological tissue, a cone-shaped finger phantom (**Fig.2**) was immersed in 2 liters of water as the scanning subject. The data acquisition was conducted using the DeskCAT Multi-slice Optical CT Scanner. A collimator mask with a horizontal slit was installed in front of the camera to restrict the detection range to the central slice only.

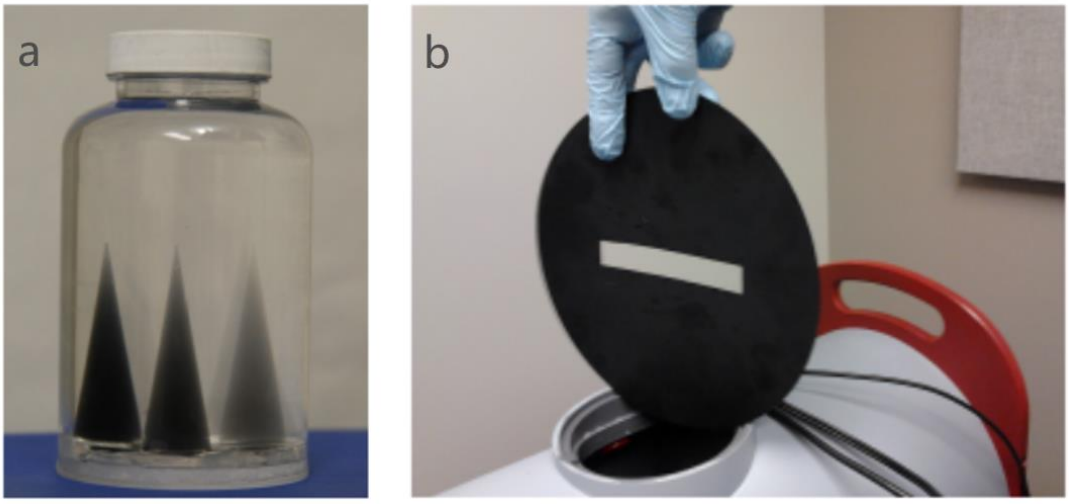

Fig 2. *Cone-shaped finger phantom and collimator mask.*

Before actual scanning, a geometry calibration must be performed without the phantom. This was done by selecting "Calibration → Geometry Calibration → Auto-Cal" in the software interface and accepting the automatically provided values.

Next, the camera brightness was adjusted to 50% of its maximum value to help create more evenly distributed noise and simulate different exposure levels. This adjustment was done via "Scanner → Camera Settings."

After these settings, the number of projections was set using the side panel, and reference scan data (background scan without phantom) was collected by clicking the appropriate button.

The cone-shaped phantom was then secured onto the rotary stage using a jar clamp and mounted into the scanner. The data scan was initiated by pressing "Start Data Scan," and the system automatically performed the full acquisition process.

2.2 Dataset for SNR and CNR analysis

The rawdata obtained from the steps mention above contains the following parts:

- ScanData: Projection data collected after placing the phantom. ScanRef: Projection data collected without the phantom in place. Data in ScanData and ScanRef folder contains 640 projection pictures with size of 480*640*3 in BMP format.
- Calibration.xml: Geometric parameters.
- Info.xml: Contains the number of projections in ScanData and ScanRef.

The sinogram of ScanData in the index 240 is shown in Fig.3.

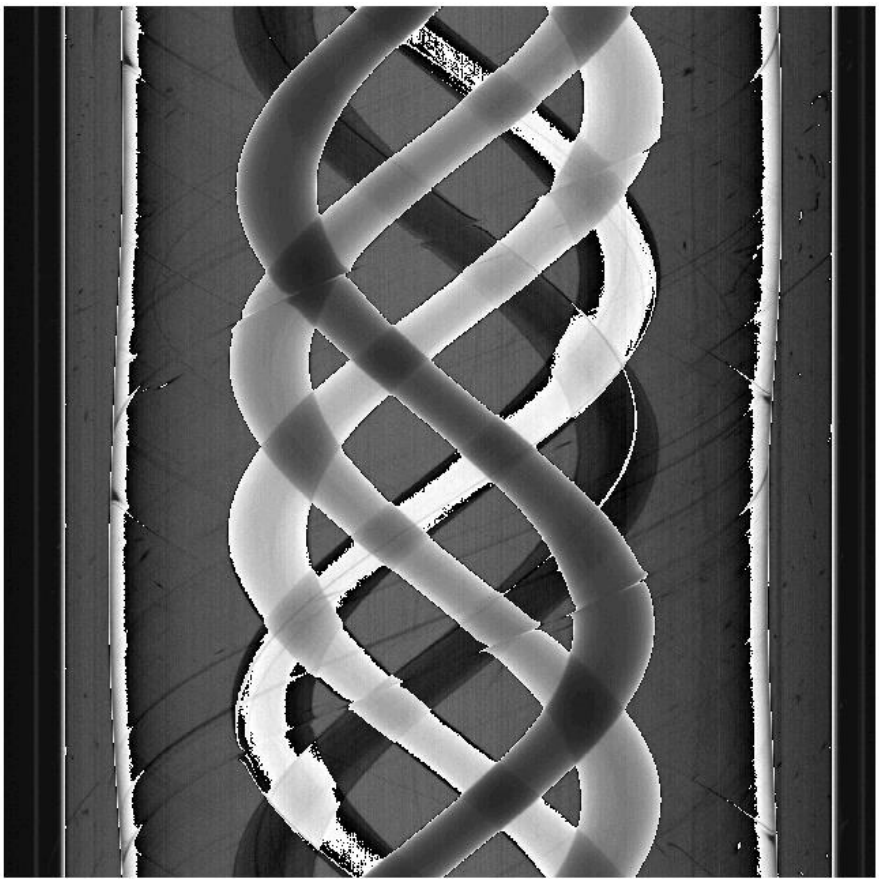

Fig 3. *Sinogram of phantom scan*

2.3 Image reconstruction using DeskCAT

After data collection, the Hamming filter was selected under "Reconstruction → Reconstruction Options." Voxel resolution was configured, and the reconstruction process was started by pressing "Start Reconstruction."

The images reconstructed by DeskCAT (**Fig.3**) were visualized through the software interface to assess image quality and provide a foundation for subsequent analysis.

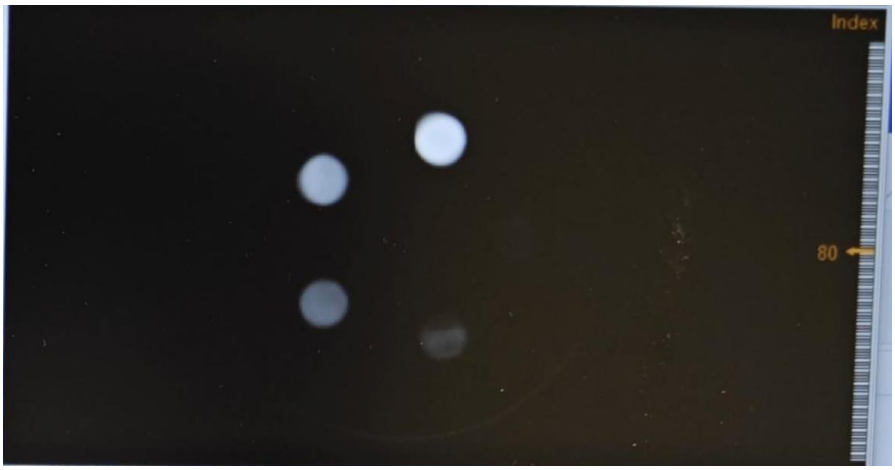

Fig 4. *Images reconstructed by DeskCAT*

3. RECONSTRUCTION

In this project, we employed a filtered back-projection (FBP) method for image reconstruction from fan-beam CT projection data. Prior to reconstruction, we applied flat-field correction to compensate for variations in detector sensitivity and source intensity, ensuring more accurate measurements. The corrected projection data were then converted to a logarithmic scale to better represent X-ray attenuation through the scanned object.

The projection data were acquired using a full-scan fan-beam geometry, where the X-ray source rotates around the object while a flat-panel detector captures projections at multiple angles. This setup enables the reconstruction of high-resolution cross-sectional images with complete angular coverage.

To reconstruct the images, we used the FBP algorithm, which is a widely adopted analytical approach in CT due to its computational efficiency and ease of implementation. It combines the use of filtering to enhance projection data and back-projection to synthesize the final image. In our case, a commonly used filter was applied to reduce noise and improve the quality of the reconstructed images.

This method allowed us to obtain clear and detailed reconstructions of the scanned object, demonstrating the effectiveness of fan-beam CT in conjunction with standard correction and

reconstruction techniques.

Reconstruction uses an algorithmic implementation of the ASTRA toolkit. The reconstructed image using the method mentioned above is shown in Fig.4.

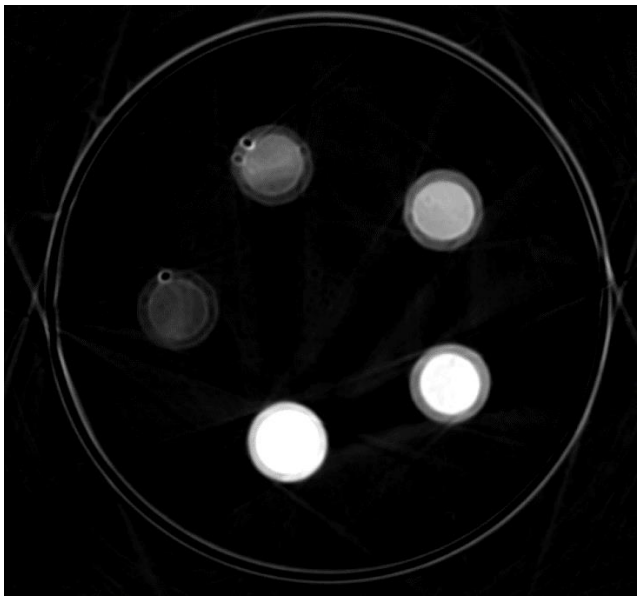

Fig 5. Reconstructed image using FBP method

4. ANALYSIS

To explore the role and principles of CNR and SNR in evaluating image quality in CT scans. The relevant ROIs should be selected carefully, like the regions should focus on the concentration and avoid the obvious noise interference. Beyond, the ROI 1 has been selected as the reference with the highest SNR, and compare the SNR and CNR of various ROIs.

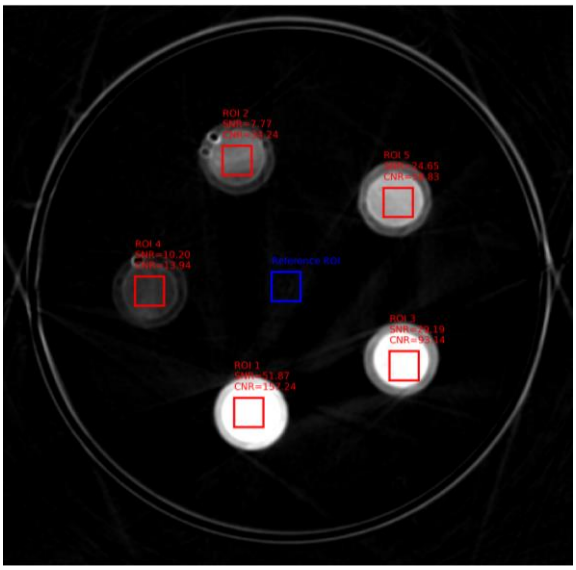

Fig 6. ROI selected from the reconstructed image

We use the python to calculate the means and std of the different ROIs, the results are shown below:

Tab. 1 Mean and Std of different ROI

Region	Means	Std
Ref_ROI	0.8266	0.2911
ROI 1	46.6019	0.8985
ROI 2	10.7955	1.2893
ROI 3	27.9398	0.9572
ROI 4	4.8852	0.4792
ROI 5	17.9537	0.7283

As two Formula has be mentioned in background part, we can use these (1) and (2) to calculate the SNR and CNR of the different ROIs.

Tab. 2 SNR and CNR in Each Region

Region	SNR	CNR(Reference: Background)	CNR(Reference: ROI 1)
ROI 1	160.0890	48.4662	0
ROI 2	37.0852	7.5422	27.0901
ROI 3	95.9801	27.1000	18.6530
ROI 4	16.7879	7.2386	74.4026
ROI 5	61.6754	21.8368	36.5261

These results show that as the metering decreases, both the SNR and CNR of the ROI show a downward trend.

5. ADDITIONAL QUESTION

Question: By using different camera exposure times, you can obtain raw data with different amounts of photons (i.e. ‘radiation’ doses). How will CNR and SNR change with ‘radiation’ dose?

Answer:

In imaging, radiation dose is typically proportional to the number of photons. The longer the exposure time, the more photons are emitted, and the higher the dose.

First, consider SNR. According to statistical physics, noise generally follows a Poisson distribution, where the relationship between the signal mean (μ) and the noise standard deviation (σ) is $\sigma = \sqrt{\mu}$. Thus, SNR = μ/σ. This means SNR is proportional to the square root of the number of photons. As the radiation dose increases (more photons), SNR improves, but the rate of improvement gradually slows down, following a square-root relationship.

Next, consider CNR. As the dose increases, σ decreases (since μ increases and $\sigma = \sqrt{\mu}$), leading to an improvement in CNR.

In summary, as the radiation dose increases (longer exposure time, more photons), both SNR and CNR improve, but only at a square-root rate. In practical applications, there is a trade-off: increasing the dose enhances image quality but also raises radiation risks. Therefore, it is necessary to optimize the exposure time to balance diagnostic needs and safety.

Appendix

```
import os
import numpy as np
import xml.etree.ElementTree as ET
import matplotlib.pyplot as plt
import astra
from scipy.ndimage import gaussian_filter
import pandas as pd

# ------------------------------
# Configuration
scan_data_dir = 'ScanData'
scan_ref_dir = 'ScanRef'
calib_path = 'calibration.xml'
info_path = 'info.xml'
header_bytes = 54

# BMP image dimensions
bmp_height = 480  # number of rows
```

```python
bmp_width = 640  # number of detector channels


# ------------------------------
def parse_xml(calib_file, info_file):
    """Parse XML and return geometric parameters."""
    calib = ET.parse(calib_file).getroot()
    info = ET.parse(info_file).getroot()

    # distances in mm
    d_sod_mm = float(calib.find('SourceToAxis').text)
    d_odd_mm = float(calib.find('AxisToDetector').text)
    px_sz_mm = float(calib.find('HorizPixelSize').text)
    axis_off_mm = float(calib.find('AxisOfRotationOffset').text)
    num_proj = int(info.find('DataProjections').text)

    # convert axis offset to pixels
    axis_off_px = axis_off_mm / px_sz_mm

    return d_sod_mm, d_odd_mm, px_sz_mm, axis_off_px, num_proj


def load_projection_line(fp):
    """Load a 16-bit BMP, reshape to (480,640), return its central row."""
    with open(fp, 'rb') as f:
        buf = f.read()
    arr = np.frombuffer(buf, dtype=np.uint16, offset=header_bytes)
    arr = arr[:bmp_height * bmp_width]
    img = arr.reshape((bmp_height, bmp_width))
    return img[bmp_height // 2, :].astype(np.float32)


def calculate_snr_and_cnr(image, roi_coords, reference_roi):
    """Calculate SNR and CNR for given ROIs."""
    snr_values = []
    cnr_values = []
    roi_means = []
    roi_stds = []

    # Ensure ROIs are within image bounds
    height, width = image.shape
    ref_x, ref_y, ref_w, ref_h = reference_roi
    ref_x = max(0, min(ref_x, width - 1))
    ref_y = max(0, min(ref_y, height - 1))
```

```python
    ref_w = min(ref_w, width - ref_x)
    ref_h = min(ref_h, height - ref_y)

    if ref_w <= 0 or ref_h <= 0:
        raise ValueError(f"Reference ROI dimensions invalid: w={ref_w}, h={ref_h}")

    # Extract reference ROI (background)
    ref_roi = image[ref_y:ref_y + ref_h, ref_x:ref_x + ref_w]
    mu_b = np.mean(ref_roi)
    sigma_b = np.std(ref_roi)

    print(f"Reference ROI ({ref_x},{ref_y},{ref_w},{ref_h}): mean = {mu_b:.4f}, std = {sigma_b:.4f}")

    for i, roi in enumerate(roi_coords):
        x, y, w, h = roi
        # Ensure ROI is within image bounds
        x = max(0, min(x, width - 1))
        y = max(0, min(y, height - 1))
        w = min(w, width - x)
        h = min(h, height - y)

        if w <= 0 or h <= 0:
            print(f"Warning: ROI {i + 1} dimensions invalid, skipping")
            snr_values.append(float('nan'))
            cnr_values.append(float('nan'))
            roi_means.append(float('nan'))
            roi_stds.append(float('nan'))
            continue

        roi_data = image[y:y + h, x:x + w]
        mu_o = np.mean(roi_data)
        sigma_o = np.std(roi_data)

        roi_means.append(mu_o)
        roi_stds.append(sigma_o)

        print(f"ROI {i + 1} ({x},{y},{w},{h}): mean = {mu_o:.4f}, std = {sigma_o:.4f}")

        # Calculate SNR
        snr = mu_o / sigma_o if sigma_o > 0 else float('inf')
        snr_values.append(snr)
```

```python

        # Calculate CNR
        cnr = abs(mu_o - mu_b) / sigma_b if sigma_b > 0 else float('inf')
        cnr_values.append(cnr)

    return snr_values, cnr_values, roi_means, roi_stds


def get_centered_roi(image, center_x, center_y, size=30):
    """Get ROI coordinates centered at a point"""
    h, w = image.shape
    half_size = size // 2
    x = max(0, min(center_x - half_size, w - size))
    y = max(0, min(center_y - half_size, h - size))
    return (x, y, size, size)


def main():
    # 1. Parse geometry
    d_sod_mm, d_odd_mm, px_sz_mm, axis_off_px, num_proj = parse_xml(calib_path, info_path)
    print(
        f"Projections={num_proj}, SOD={d_sod_mm}mm, ODD={d_odd_mm}mm, px_sz={px_sz_mm}mm, axis_off={axis_off_px:.1f}px")

    # 2. Build flat-field line
    ref_files = sorted([f for f in os.listdir(scan_ref_dir) if f.lower().endswith('.bmp')])[:num_proj]
    flats = [load_projection_line(os.path.join(scan_ref_dir, f)) for f in ref_files]
    flat_line = np.mean(flats, axis=0)
    flat_line = np.clip(flat_line, 1e-6, None)

    # 3. Load projections, correct and log-transform → sinogram
    data_files = sorted([f for f in os.listdir(scan_data_dir) if f.lower().endswith('.bmp')])[:num_proj]
    sinogram = np.zeros((num_proj, bmp_width), dtype=np.float32)
    for i, fname in enumerate(data_files):
        proj = load_projection_line(os.path.join(scan_data_dir, fname))
        proj = np.clip(proj, 1e-6, None)
        corr = proj / flat_line
        corr = np.clip(corr, 1e-8, None)
        sinogram[i] = -np.log(corr)

```

```python
# visualize sinogram
plt.figure(figsize=(8, 6))
plt.imshow(sinogram, cmap='gray', aspect='auto')
plt.title('Sinogram (Full-scan Fan-beam)')
plt.xlabel('Detector pixel')
plt.ylabel('Projection index')
plt.colorbar()
plt.tight_layout()
plt.savefig('sinogram_fullscan.png', dpi=300)

# 4. Set up ASTRA full-scan fan-beam geometry
angles = np.linspace(0, 2 * np.pi, num_proj, endpoint=False)
proj_geom = astra.create_proj_geom('fanflat',
                                   px_sz_mm,  # detector spacing [mm]
                                   bmp_width,
                                   angles,
                                   d_sod_mm,
                                   d_odd_mm)

# 5. Create volume geometry with physical extents in mm
xmin = -0.5 * bmp_width * px_sz_mm
xmax = 0.5 * bmp_width * px_sz_mm
ymin = -0.5 * bmp_height * px_sz_mm
ymax = 0.5 * bmp_height * px_sz_mm
vol_geom = astra.create_vol_geom(bmp_height, bmp_width,
                                 xmin, xmax,
                                 ymin, ymax)

# 6. Create ASTRA data and run FBP
sino_id = astra.data2d.create('-sino', proj_geom, sinogram)
rec_id = astra.data2d.create('-vol', vol_geom, 0)

try:
    cfg = astra.astra_dict('FBP_CUDA')
    print("Using GPU FBP_CUDA")
except:
    cfg = astra.astra_dict('FBP')
    print("Using CPU FBP")

cfg.update(ProjectionDataId=sino_id,
           ReconstructionDataId=rec_id,
           option={'FilterType': 'Hamming',
                   'ParkerWeighting': False})

```

```python
    alg_id = astra.algorithm.create(cfg)
    astra.algorithm.run(alg_id)
    rec = astra.data2d.get(rec_id)

    # 7. Optional denoising
    denoised = gaussian_filter(rec, sigma=1.0)

    # 8. Display reconstructions
    fig, axes = plt.subplots(1, 2, figsize=(12, 6))
    axes[0].imshow(rec, cmap='gray', vmin=0, vmax=np.percentile(rec, 99))
    axes[0].set_title('Raw Reconstruction')
    axes[0].axis('off')
    axes[1].imshow(denoised, cmap='gray', vmin=0, vmax=np.percentile(denoised, 99))
    axes[1].set_title('Denoised (σ=1)')
    axes[1].axis('off')
    plt.tight_layout()
    plt.savefig('reconstruction_fullscan.png', dpi=300)

    # 打印图像尺寸，便于设置 ROI
    print(f"Image shape: {denoised.shape}")

    # 9. Define ROIs based on image center
    height, width = denoised.shape
    center_x, center_y = width // 2, height // 2

    # 选择合适的 ROI 位置（围绕图像中心）
    roi_size = 25
    roi_coords = [
        # 白色样本 ROI (根据实际图像调整坐标)
        get_centered_roi(denoised, center_x - width//20, center_y + height*9//40, roi_size),
        get_centered_roi(denoised, center_x - width//15, center_y - height*9//40, roi_size),
        get_centered_roi(denoised, center_x + width*19//120, center_y + height//7, roi_size),
        get_centered_roi(denoised, center_x - width*11//60, center_y + height//100, roi_size),
        get_centered_roi(denoised, center_x + width*9//60, center_y - height*9//60, roi_size),
        ]

    # 背景 ROI 选择在没有样本的区域
    reference_roi = get_centered_roi(denoised, center_x, center_y, roi_size)
```

```python

    # 10. Calculate SNR and CNR with clear error reporting
    print("\n===== SNR & CNR Calculation =====")
    try:
        snr_values, cnr_values, roi_means, roi_stds = calculate_snr_and_cnr(denoised, roi_coords, reference_roi)

        # 直接打印关键结果，确保值显示出来
        print("\n----- Raw SNR & CNR Values -----")
        for i in range(len(roi_coords)):
            print(f"ROI {i + 1}: SNR = {snr_values[i]:.4f}, CNR = {cnr_values[i]:.4f}")

        # 11. Print results in table format
        print("\n===== Results Table =====")
        results = pd.DataFrame({
            'ROI': [f'ROI {i + 1}' for i in range(len(roi_coords))],
            'Mean': roi_means,
            'Std Dev': roi_stds,
            'SNR': snr_values,
            'CNR': cnr_values
        })
        print(results)

        # 保存结果到 CSV
        results.to_csv('snr_cnr_results.csv', index=False)
        print("Saved results to snr_cnr_results.csv")

        # 12. Display results with ROIs
        plt.figure(figsize=(10, 8))
        plt.imshow(denoised, cmap='gray', vmin=0, vmax=np.percentile(denoised, 99))

        # 添加 ROI 标记
        for i, (x, y, w, h) in enumerate(roi_coords):
            plt.gca().add_patch(plt.Rectangle((x, y), w, h, edgecolor='red', facecolor='none', lw=2))
            plt.text(x, y - 5, f"ROI {i + 1}\nSNR={snr_values[i]:.2f}\nCNR={cnr_values[i]:.2f}",
                     color='red', fontsize=9, verticalalignment='bottom')

        # 添加参考 ROI 标记
        ref_x, ref_y, ref_w, ref_h = reference_roi
```

```python
        plt.gca().add_patch(plt.Rectangle((ref_x, ref_y), ref_w, ref_h, edgecolor='blue', facecolor='none', lw=2))
        plt.text(ref_x, ref_y - 5, "Reference ROI", color='blue', fontsize=9, verticalalignment='bottom')

        plt.title("ROIs with SNR & CNR Values")
        plt.axis('off')
        plt.tight_layout()
        plt.savefig('rois_snr_cnr.png', dpi=300)

    except Exception as e:
        import traceback
        print(f"Error calculating SNR/CNR: {e}")
        traceback.print_exc()

        # 尝试使用更简单的方法计算和显示
        print("\n----- Attempting simplified SNR & CNR calculation -----")
        # 使用简化的计算方法
        try:
            # 重新定义一个更简单的 ROI 中心位置
            simple_roi = [(width // 4, height // 4, 20, 20)]  # 简单的 ROI 位置
            simple_ref = (3 * width // 4, 3 * height // 4, 20, 20)  # 简单的参考 ROI

            # 提取 ROI 区域
            roi_data = denoised[height // 4:height // 4 + 20, width // 4:width // 4 + 20]
            ref_data = denoised[3 * height // 4:3 * height // 4 + 20, 3 * width // 4:3 * width // 4 + 20]

            # 计算
            roi_mean = np.mean(roi_data)
            roi_std = np.std(roi_data)
            ref_mean = np.mean(ref_data)
            ref_std = np.std(ref_data)

            # SNR 和 CNR
            simple_snr = roi_mean / roi_std if roi_std > 0 else float('inf')
            simple_cnr = abs(roi_mean - ref_mean) / ref_std if ref_std > 0 else float('inf')

            print(f"Simple ROI: mean = {roi_mean:.4f}, std = {roi_std:.4f}")
            print(f"Simple Ref: mean = {ref_mean:.4f}, std = {ref_std:.4f}")
            print(f"Simple SNR = {simple_snr:.4f}, CNR = {simple_cnr:.4f}")
```

```python

            # 确保这些值在命令行中打印出来
            import sys
            sys.stdout.flush()
        except Exception as e2:
            print(f"Simplified calculation also failed: {e2}")

    # 13. Save and clean up
    np.save('reconstruction_fullscan.npy', rec)
    print("Saved reconstruction_fullscan.npy and .png outputs")

    # 这里再打印一次 SNR/CNR 结果以确保它们显示出来
    try:
        print("\n===== FINAL SNR & CNR VALUES =====")
        for i in range(len(roi_coords)):
            print(f"ROI {i + 1}: SNR = {snr_values[i]:.4f}, CNR = {cnr_values[i]:.4f}")
    except:
        print("Could not print final SNR & CNR values")

    astra.algorithm.delete(alg_id)
    astra.data2d.delete(sino_id)
    astra.data2d.delete(rec_id)


if __name__ == '__main__':
    main()
```